Towards AI · Following

Member-only story

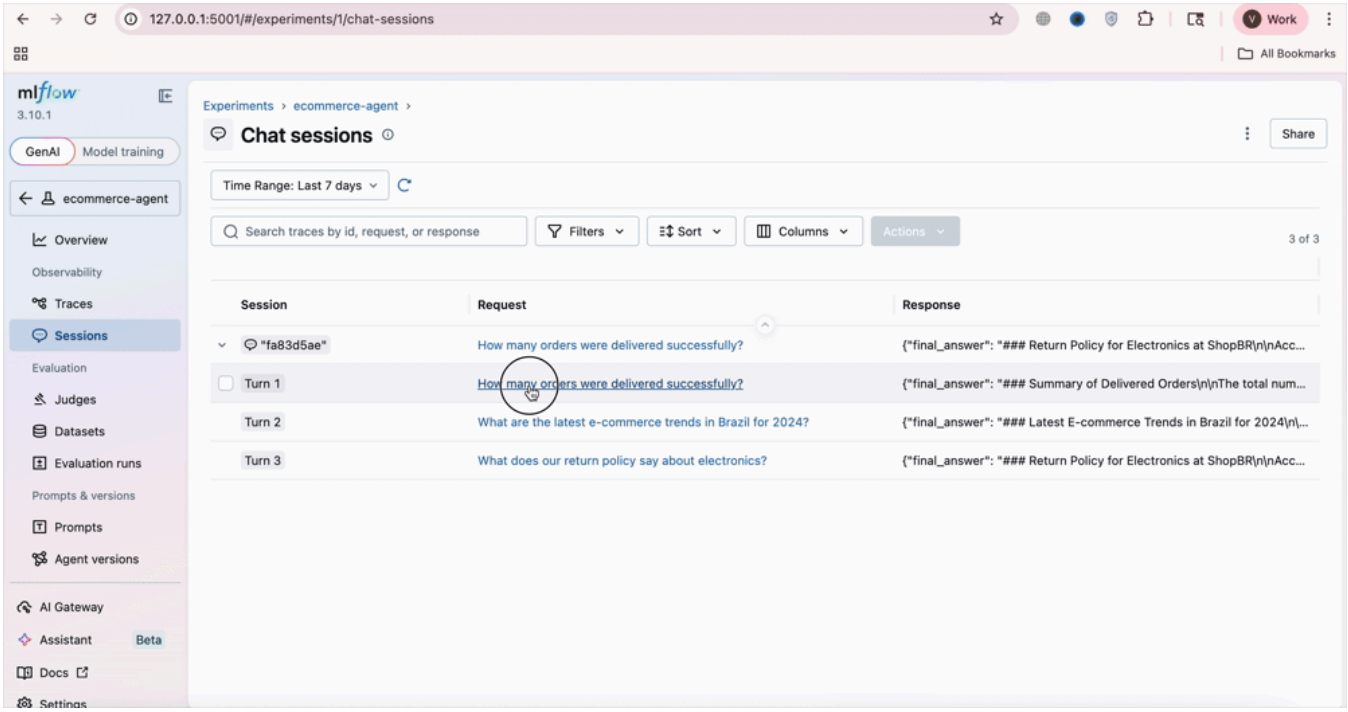
Agentic AI Project: MLflow Observability for Generative AI - A Deep Dive with Text2SQL + RAG + WebSearch using LangGraph

Because your agents are failing and you just can't see where. And tracing LLM calls alone won't explain why?

38 min read · Apr 11, 2026

Alpha Iterations Follow

Listen Share More



MLFLow Implementation

Non members, read here for free.

The Invisible Problem in Production GenAI

You've built a sophisticated GenAI or Agentic AI pipeline. It works beautifully in your notebook or repo. Then it ships to production, and three weeks later a stakeholder messages you:

"Why did the chatbot give a completely wrong answer to a simple question about orders last Tuesday at 2pm?"

You open your logs. You find… timestamps and HTTP status codes.

Sound familiar?

This is the core problem with GenAI systems:

they fail silently, expensively, and in ways that are fundamentally different from traditional software.**

> *A REST API either returns a 200 or it doesn't. An LLM returns a 200* **and** *a confidently wrong answer.*

Tracking latency alone tells you nothing about whether the model hallucinated. Logging prompts without context tells you nothing about which retrieval step went wrong.

MLflow 2.x introduced a native tracing system designed specifically for this world: multi-hop LLM calls, RAG pipelines, agents with tool use. In this article, we'll build a production-grade **Text2SQL + RAG pipeline + WebSearch** using LangGraph and the OpenAI API, instrument it fully with MLflow, and explore what real observability looks like for GenAI systems.

What Makes GenAI Observability Different?

Traditional ML observability focuses on model drift, data distribution shifts, and accuracy metrics computed offline. GenAI observability has a completely different shape:

Concern	Traditional ML	GenAI
Latency	Model inference time	Multi-hop API call chains
Failure modes	Prediction errors, null outputs	Hallucinations, context loss, tool misuse
Cost	Compute per batch	Token usage per request
Quality	Accuracy, AUC, F1	LLM-as-judge, faithfulness, relevance
Debugging unit	Feature vector → prediction	Full conversation trace

Observability: Traditional ML vs GenAI

MLflow's tracing system addresses all of these with three core primitives:

- **Spans:** Named, timed units of work (a retrieval call, an LLM call, a SQL execution)
- **Traces:** Hierarchical trees of spans representing a full request lifecycle
- **Evaluations:** Structured metric logging using LLM-as-judge or custom scorers

The Three Core Primitives of MLflow Tracing

At the heart of MLflow's GenAI observability system are three deceptively simple concepts: **Spans, Traces and Evaluations.**

Individually, they're straightforward. Together, they give **A complete, inspectable execution story of every single request.**

1. Spans: The Building Blocks of Execution

Everything starts with a **span.**
In GenAI systems, spans map to conceptual operations: a retrieval from a vector store, an LLM completion call, a SQL execution, or a tool invocation.

A span represents a **single, well-defined unit of work** inside your pipeline.
It has:

- A **name** (what is happening)
- A **start and end time** (how long it took)
- **Inputs and outputs** (what went in, what came out)

- Optional **metadata** (tokens, model, parameters, errors)

Span Type	Purpose	Typical Attributes
CHAIN	Orchestration logic, agent loops	`conversation_turns`, `agent_type`
LLM / CHAT_MODEL	Language model inference	`token_usage`, `model_name`, `temperature`, `cost_usd`
RETRIEVER	Vector search, document fetch	`num_documents`, `query_embedding`, `top_k_distance`
TOOL	External function execution	`tool_name`, `execution_time_ms`, `error_type`
PARSER	Output parsing, validation	`parser_type`, `validation_success`

Types of SPANs in MLFlow

In traditional systems, this might feel like a function call.
In GenAI systems, spans become much more critical, as each step can independently fail *semantically*, not just technically.

2. Traces: The Story of a Single Request

A trace is the complete tree of spans for a single user request.

While spans answer “what happened in this step?”, traces answer “what happened in this conversation turn?”

Every time a user sends a query, MLflow creates one trace that ties everything together.

The spans don’t exist in isolation. They are connected:

```
Trace: user_query_request
│
├── Span: parse_query_llm
├── Span: retrieve_context
├── Span: generate_sql_llm
├── Span: execute_sql
└── Span: summarize_response_llm
```

This structure gives us:

- End-to-end latency
- Execution order
- Parent-child relationships

- Full context propagation

3. Evaluations: Quality at Scale

Spans and traces tell you *what happened*. **Evaluations tell you whether it was good.** This is where GenAI observability diverges completely from traditional systems.

Because in GenAI:

- A request can succeed technically
- And still fail **semantically**

MLflow supports two evaluation modes:

Online (Per-Trace) Feedback

This captures **real-time, request-level evaluations** tied to individual traces. Feedback can come from humans (e.g., ratings, annotations) or automated LLM judges, helping you assess quality for each interaction as it happens.

Offline (Batch) Evaluation

This runs **structured evaluations across datasets** to compute aggregate metrics. It's typically used for regression testing, benchmarking, and comparing different models or prompt strategies at scale.

We will keep the evaluation out of scope for this article during our implementation.

Let's see these in action with a real system.

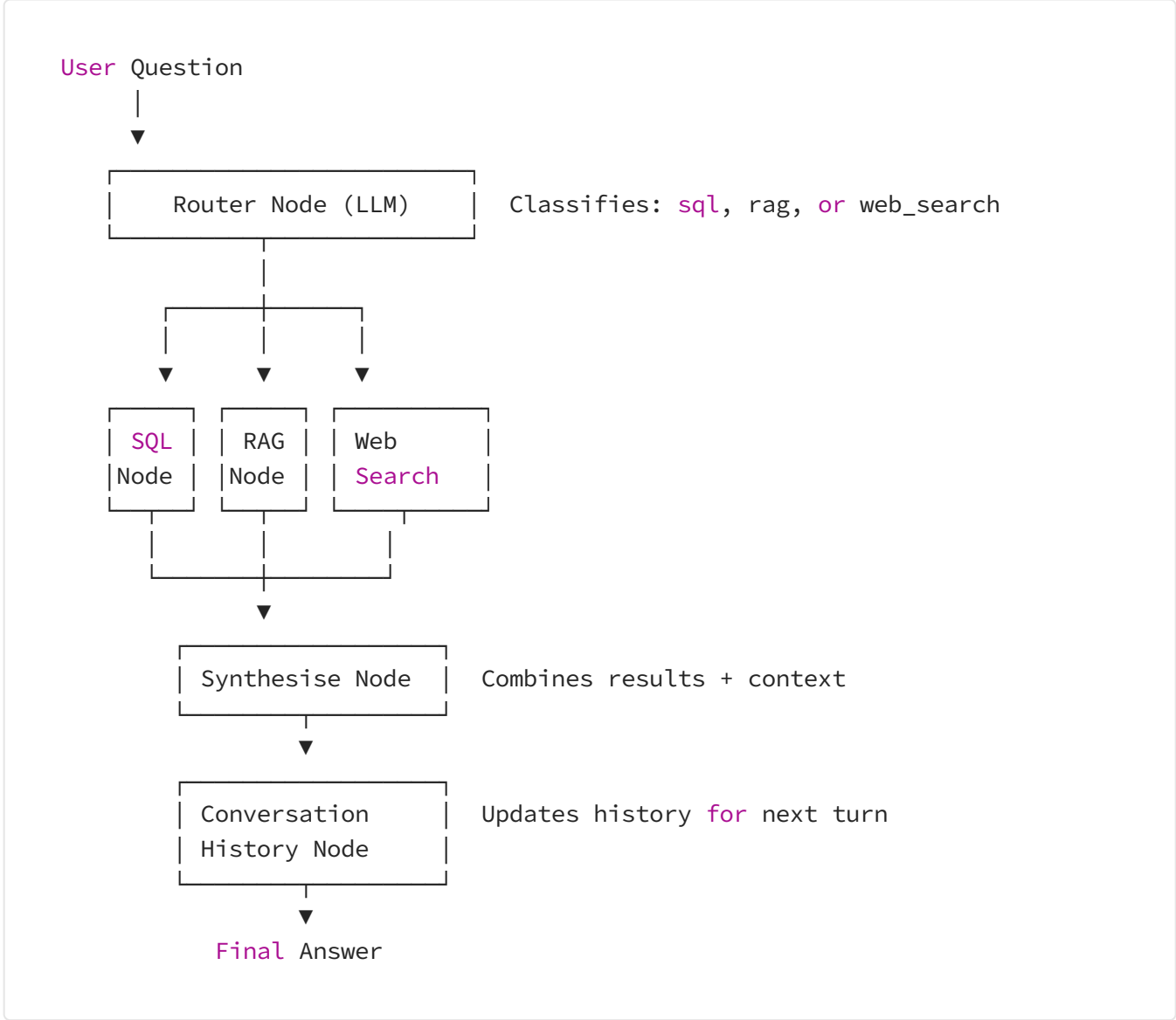
The Case Study: E-Commerce Conversational Analytics

We are building an e-commerce conversational agent that:
- routes each user question to the best tool,
- answers analytical SQL questions from a local SQLite database,
- retrieves relevant text from uploaded PDFs using FAISS vector search,
- performs live web search using Serper when external information is needed,
- logs observability traces with MLflow for every step.

The agent is implemented as a LangGraph state machine with clearly separated nodes for routing, tools, synthesis, and conversation history.

1. Architecture Overview: The Observable Agent

The agent follows a clear, observable state machine pattern:



Each node in this graph becomes a trace span in MLflow, giving you hierarchical visibility.

Let’s begin now.

Complete Code is open sourced here.

2. Prerequisites

Before you begin, make sure you have:

1. Python 3.11 or newer installed.

2. `pip` available.

3. A working internet connection for data download and API calls.

4. OpenAI API key.

5. Serper API key for web search.

You can clone the repository.

The repository already includes the main source files and configuration:

```
├── .env
├── .gitignore
├── README.md
├── agent
│   ├── __init__.py
│   ├── graph.py
│   ├── nodes.py
│   ├── router.py
│   └── state.py
├── config.py
├── data
│   ├── ecommerce.db
│   ├── faiss_index
│   │   ├── index.faiss
│   │   └── metadata.pkl
│   └── raw
│       ├── df_Customers.csv
│       ├── df_OrderItems.csv
│       ├── df_Orders.csv
│       ├── df_Payments.csv
│       └── df_Products.csv
├── data_loader.py
├── main.py
├── medium_tutorial.md
├── mlflow.db
├── observability.py
├── pdf_docs
│   └── shopbr_return_policy.pdf
├── requirements.txt
├── session.py
├── setup.py
└── tools
    ├── __init__.py
```

```
    ├── rag_tool.py
    ├── sql_tool.py
    └── web_search_tool.py
```

All files are part of the repo except the `.csv` and .db files. The `.csv` files can be downloaded using setup.py file explained in the later section of the article.

3. Setup Step-by-Step

Step 1: Create and Activate Virtual Environment

For macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

For Windows (Command Prompt):**

```
python -m venv .venv
.venv\Scripts\activate
```

Step 2: Install Python dependencies

Install the required libraries from `requirements.txt`.

```
pip install -r requirements.txt
```

This installs the packages that power the agent, including:

```
- openai
- langgraph
- faiss-cpu
- sentence-transformers
- pypdf
- pandas
- kagglehub
```

```
  - mlflow
  - python-dotenv
```

Step 3: Configure your API keys

Please add a .env file and update below environment variables.

```
OPENAI_API_KEY=your-open-api-key
SERPER_API_KEY=your-serper-api-key
MLFLOW_TRACKING_URI=http://localhost:5001
MLFLOW_EXPERIMENT=ecommerce-agent
```

4. Load the Data

4.1: E-Commerce Data

Source: Kaggle: Ecommerce Order & Supply Chain Dataset

The dataset used in this project is the **E-commerce Order Dataset** available on Kaggle. It provides a **multi-table view of an end-to-end e-commerce system**, covering orders, customers, products, payments, and logistics.

At a high level, this dataset simulates the **entire lifecycle of an order**, from the moment a customer places it to final delivery.

Key Tables in the Dataset

1. Orders

Contains core order lifecycle information:

- Order ID, customer ID
- Order status (delivered, canceled, etc.)
- Purchase, approval, and delivery timestamps
- Estimated delivery date

👉 Useful for analyzing delivery delays, order flow, and lifecycle tracking.

2. Order Items

Represents items within each order:

- Product ID and seller ID
- Price and shipping charges
- Multiple items per order supported

👉 Critical for revenue analysis and basket-level insights.

3. Customers
Customer-level metadata:

- Customer ID
- Location (city, state, zip code)

👉 Enables geographic analysis and segmentation.

4. Payments
Payment transaction details:

- Payment method (credit card, etc.)
- Installments
- Payment value

👉 Useful for understanding payment behavior and success rates.

5. Products
Product catalog information:

- Product category
- Physical attributes (weight, dimensions)

👉 Enables category-level insights and logistics analysis.

```
# data_loader.py
# ──────────────────────────────────────────────────────────────────────
# Downloads the Kaggle e-commerce dataset and ingests the train/ CSVs into
# a local SQLite database.
#
# Table mapping (Kaggle train/ folder → SQLite table):
#   orders_dataset.csv       → orders
```

```
#   order_items_dataset.csv  → order_items
#   customers_dataset.csv    → customers
#   payments_dataset.csv     → payments  (olist_order_payments_dataset.csv)
#   products_dataset.csv     → products
#
# Usage:
#   python data_loader.py
# ──────────────────────────────────────────────────────────────────────────

import os
import sqlite3
import glob
import shutil
import pandas as pd
import kagglehub
from config import DB_PATH, RAW_DATA_DIR

# ── File name fragments to detect each table ──────────────────────────────
TABLE_HINTS = {
    "orders":      ["orders_dataset", "orders.csv"],
    "order_items": ["order_items_dataset", "orderitems.csv"],
    "customers":   ["customers_dataset", "customers.csv"],
    "payments":    ["payments_dataset", "payment"],
    "products":    ["products_dataset", "products.csv"],
}


def _find_csv(train_dir: str, hints: list[str]) -> str | None:
    """Return the first CSV in train_dir whose name contains any of hints."""
    for f in glob.glob(os.path.join(train_dir, "*.csv")):
        fname = os.path.basename(f).lower()
        if any(h.lower() in fname for h in hints):
            return f
    return None


def _resolve_delivery_col(df: pd.DataFrame) -> pd.DataFrame:
    """Kaggle orders files may use different column names for delivery date."""
    if "order_delivered_timestamp" in df.columns:
        return df
    for candidate in ("order_delivered_customer_date", "order_delivered_carrier_
        if candidate in df.columns:
            df = df.rename(columns={candidate: "order_delivered_timestamp"})
            return df
    return df


def _copy_raw_csvs(source_dir: str, target_dir: str) -> None:
    """Copy raw CSVs from the downloaded dataset into the local raw data folder.
    csv_files = glob.glob(os.path.join(source_dir, "*.csv"))
    if not csv_files:
        print(f"[data_loader] WARNING: No CSV files found in {source_dir} to cop
        return
```

```
        for csv_path in csv_files:
            target_path = os.path.join(target_dir, os.path.basename(csv_path))
            shutil.copy2(csv_path, target_path)


    def download_and_ingest(force: bool = False) -> str:
        """
        Download the Kaggle dataset (cached after first run) and write all tables
        into the SQLite database at DB_PATH.

        Returns DB_PATH.
        """
        os.makedirs(os.path.dirname(DB_PATH), exist_ok=True)
        os.makedirs(RAW_DATA_DIR, exist_ok=True)

        if os.path.exists(DB_PATH) and not force:
            print(f"[data_loader] SQLite DB already exists at {DB_PATH}. Skipping in
            print("              Pass force=True to re-ingest.")
            return DB_PATH

        print("[data_loader] Downloading Kaggle dataset …")
        kaggle_root = kagglehub.dataset_download("bytadit/ecommerce-order-dataset")
        print(f"[data_loader] Dataset root: {kaggle_root}")

        # Find the train/ sub-directory
        train_dir = None
        for root, dirs, _ in os.walk(kaggle_root):
            if "train" in dirs:
                train_dir = os.path.join(root, "train")
                break
            # Some versions have the CSVs directly under kaggle_root/train
            if os.path.basename(root).lower() == "train":
                train_dir = root
                break

        if train_dir is None:
            # Fallback: use root directly if no train sub-folder found
            train_dir = kaggle_root
        print(f"[data_loader] Using CSV directory: {train_dir}")

        print(f"[data_loader] Copying raw CSVs to {RAW_DATA_DIR} …")
        _copy_raw_csvs(train_dir, RAW_DATA_DIR)

        conn = sqlite3.connect(DB_PATH)

        for table_name, hints in TABLE_HINTS.items():
            csv_path = _find_csv(train_dir, hints)
            if csv_path is None:
                print(f"[data_loader] WARNING: Could not find CSV for table '{table_
                continue

            print(f"[data_loader] Loading {os.path.basename(csv_path)} → table '{tab
```

```
            df = pd.read_csv(csv_path, low_memory=False)

            # Normalise column names: lowercase, strip whitespace
            df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]

            # Fix delivery column name for orders table
            if table_name == "orders":
                df = _resolve_delivery_col(df)

            df.to_sql(table_name, conn, if_exists="replace", index=False)
            print(f"           → {len(df):,} rows loaded.")

    conn.close()
    print(f"[data_loader] ✓ All tables written to {DB_PATH}")
    return DB_PATH


def get_table_schema(table_name: str) -> str:
    """Return CREATE TABLE SQL for a given table (useful for prompt building)."'
    conn = sqlite3.connect(DB_PATH)
    cur  = conn.cursor()
    cur.execute(
        "SELECT sql FROM sqlite_master WHERE type='table' AND name=?",
        (table_name,),
    )
    row = cur.fetchone()
    conn.close()
    return row[0] if row else f"-- Table '{table_name}' not found"


def list_tables() -> list[str]:
    """Return all table names in the SQLite database."""
    conn = sqlite3.connect(DB_PATH)
    cur  = conn.cursor()
    cur.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name
    tables = [r[0] for r in cur.fetchall()]
    conn.close()
    return tables


if __name__ == "__main__":
    download_and_ingest()
    print("\nTables in DB:", list_tables())
    for t in list_tables():
        print(f"\n── {t} ──")
        print(get_table_schema(t))
```

4.2: PDF Data

- Repo Path : `pdf_docs/shopbr_return_policy.pdf` (Please note, this is synthetically generated pdf).

- A structured, LLM-generated policy document that outlines return, refund, and replacement rules across multiple product categories in an e-commerce platform
- Contains detailed, category-specific policies (e.g., electronics, fashion, groceries), including return windows, conditions, and exceptions
- Serves as an unstructured knowledge source for the RAG pipeline, enabling the system to answer policy-related user queries

5. Config & Observability Functions

5.1 Project Config

The `config.py` file acts as the central control layer for the entire GenAI pipeline, ensuring all components operate with consistent settings and shared configurations.

- Manages API keys, model parameters, and embedding configurations
- Defines data paths for database, PDFs, and FAISS index storage
- Configures RAG behavior such as chunk size, overlap, and top-k retrieval
- Specifies routing options across SQL, RAG, and web search tools

```
# config.py
# ─────────────────────────────────────────────────────────────────────
# Central configuration for the E-Commerce Conversational Agent
# ─────────────────────────────────────────────────────────────────────

import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

# ── API Keys ──────────────────────────────────────────────────────────
OPENAI_API_KEY  = os.getenv("OPENAI_API_KEY", "")
SERPER_API_KEY  = os.getenv("SERPER_API_KEY", "")

# ── Model ─────────────────────────────────────────────────────────────
LLM_MODEL       = "gpt-4o-mini"
LLM_TEMPERATURE = 0.0
LLM_MAX_TOKENS  = 1024

# ── Embedding model (open-source, runs locally) ──────────────────────
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"

# ── Paths ─────────────────────────────────────────────────────────────
```

```
BASE_DIR        = os.path.dirname(os.path.abspath(__file__))
DB_PATH         = os.path.join(BASE_DIR, "data", "ecommerce.db")
RAW_DATA_DIR    = os.path.join(BASE_DIR, "data", "raw")
FAISS_INDEX_DIR = os.path.join(BASE_DIR, "data", "faiss_index")
PDF_DIR         = os.path.join(BASE_DIR, "pdf_docs")

# ── RAG settings ──────────────────────────────────────────────────────
RAG_TOP_K       = 4         # chunks retrieved per query
CHUNK_SIZE      = 500       # characters per chunk
CHUNK_OVERLAP   = 80        # character overlap between chunks

# ── Routing ───────────────────────────────────────────────────────────
# The router LLM call picks one of these exact route names
ROUTES = ["sql", "rag", "web_search"]
```

5.2 Observability Layer (MLflow Tracing Wrapper)

The `observability.py` module provides a reusable tracing layer that instruments every step of the pipeline with MLflow, enabling deep visibility into execution, performance, and cost.

- Wraps functions with a `@trace` decorator to automatically create spans for tools, LLM calls, and pipelines
- Captures inputs, outputs, execution time, and errors for every span without manual logging
- Tracks token usage, estimated cost, and data size to monitor efficiency and scaling
- Enriches traces with structured metadata, making debugging and root-cause analysis significantly easier

Key Attributes Captured in Spans & Traces

These attributes are what make our observability **actually useful (not just tracing)**:

Core Execution Attributes

- `duration_ms` → Time taken by each span
- `error`, `error.message`, `error.type` → Failure tracking
- `func.name`, `func.module` → Source of execution

LLM & Cost Tracking

- `model.name` → Model used (e.g., gpt-4o-mini)
- `tokens.input`, `tokens.output`, `tokens.total` → Token usage
- `cost.usd` → Estimated cost per span

Data & Payload Tracking

- `bytes.input`, `bytes.output` → Size of inputs/outputs
- `request` → User query (auto-extracted)
- `response` → Model/tool output

Retrieval-Specific Attributes (RAG)

- `retrieval.model` → Embedding model used
- `retrieval.top_k` → Number of chunks retrieved
- `retrieval.chunks` → Number of chunks returned
- `retrieval.sources` → Number of source documents

Database / SQL Attributes

- `db.type`, `db.path` → Database metadata
- `db.rows_returned` → Number of rows fetched
- `sql` → Generated SQL query

Web Search Attributes

- `api.provider`, `api.endpoint` → External API details
- `search.top_links_count` → Number of links returned

Trace-Level Context

- `request_preview` → Short version of user query
- `response_preview` → Short version of final output

```
# observability.py
# ─────────────────────────────────────────────────────────────────────────────
# MLflow tracing with automatic cost calculation and size tracking.
#
# Auto-captured attributes:
#   - cost.usd          : Estimated cost based on tokens
#   - tokens.input      : Input token count (estimated)
#   - tokens.output     : Output token count (estimated)
#   - bytes.input       : Input size in bytes
#   - bytes.output      : Output size in bytes
#   - duration_ms       : Execution time
#
# Usage:
#   @trace(span_type="TOOL", model="gpt-4o-mini")  # Enables cost tracking
#   def run_sql_tool(...): ...
# ─────────────────────────────────────────────────────────────────────────────

import os
import functools
import time
import json
from contextlib import contextmanager
from typing import Any

import mlflow

# ── Pricing (per 1M tokens) ──────────────────────────────────────────────────
MODEL_PRICING = {
    "gpt-4o": {"input": 2.50, "output": 10.00},
    "gpt-4o-mini": {"input": 0.15, "output": 0.60},
    "gpt-4-turbo": {"input": 10.00, "output": 30.00},
    "text-embedding-3-small": {"input": 0.02, "output": 0.00},
    "text-embedding-3-large": {"input": 0.13, "output": 0.00},
}

# Rough token estimation (1 token ≈ 4 chars for English)
def estimate_tokens(text: str) -> int:
    if not text:
        return 0
    return len(text) // 4

def calculate_cost(model: str, input_tokens: int, output_tokens: int) -> float:
    pricing = MODEL_PRICING.get(model, {"input": 0.15, "output": 0.60})
    input_cost = (input_tokens / 1_000_000) * pricing["input"]
    output_cost = (output_tokens / 1_000_000) * pricing["output"]
    return round(input_cost + output_cost, 6)

# ── Setup ────────────────────────────────────────────────────────────────────

ENABLED = os.getenv("MLFLOW_ENABLED", "true").lower() == "true"
_initialized = False
```

```
    def _init():
        global _initialized
        if not _initialized and ENABLED:
            mlflow.set_tracking_uri(os.getenv("MLFLOW_TRACKING_URI", "http://localho
            mlflow.set_experiment(os.getenv("MLFLOW_EXPERIMENT", "ecommerce-agent"))
            _initialized = True

    # ── Size Calculation ──────────────────────────────────────────────────────

    def calculate_size(obj: Any) -> int:
        """Calculate approximate byte size of an object."""
        try:
            if isinstance(obj, str):
                return len(obj.encode('utf-8'))
            elif isinstance(obj, (list, dict)):
                return len(json.dumps(obj).encode('utf-8'))
            else:
                return len(str(obj).encode('utf-8'))
        except:
            return 0

    # ── The Decorator (With Cost & Size Tracking) ─────────────────────────────

    def trace(name=None, span_type="CHAIN", attributes=None, model=None):
        """
        Trace any function with automatic cost and size tracking.

        Args:
            name: Span name (defaults to function name)
            span_type: CHAIN, TOOL, RETRIEVER, PARSER, LLM
            attributes: Dict of static attributes
            model: Model name for cost tracking (e.g., "gpt-4o-mini")
        """
        static_attrs = attributes or {}

        def decorator(func):
            @functools.wraps(func)
            def wrapper(*args, **kwargs):
                if not ENABLED:
                    return func(*args, **kwargs)

                _init()
                span_name = name or func.__name__

                # Build inputs and calculate input size
                inputs = {}
                input_text_parts = []
                arg_names = func.__code__.co_varnames[:func.__code__.co_argcount]

                for i, arg_name in enumerate(arg_names):
                    if i < len(args):
                        val = args[i]
                    elif arg_name in kwargs:
```

```
                    val = kwargs[arg_name]
                else:
                    continue

                # Store truncated version for display
                display_val = val
                if isinstance(val, str):
                    input_text_parts.append(val)
                    if len(val) > 500:
                        display_val = val[:500] + "..."
                elif isinstance(val, (list, dict)):
                    input_text_parts.append(json.dumps(val))
                inputs[arg_name] = display_val

            # Explicitly set "request" as the first string argument (user_quest
            # Check positional args first, then kwargs
            request_arg = None
            for i, arg_name in enumerate(arg_names):
                if i < len(args):
                    val = args[i]
                elif arg_name in kwargs:
                    val = kwargs[arg_name]
                else:
                    continue

                # If it's a string and looks like a query/message argument, use
                if isinstance(val, str) and any(keyword in arg_name.lower() for
                    request_arg = val
                    break

            # Fallback: use first string argument if no obvious request argument
            if not request_arg:
                for i, arg_name in enumerate(arg_names):
                    if i < len(args):
                        val = args[i]
                    elif arg_name in kwargs:
                        val = kwargs[arg_name]
                    else:
                        continue
                    if isinstance(val, str):
                        request_arg = val
                        break

            if request_arg:
                inputs["request"] = request_arg

            input_size = calculate_size(args) + calculate_size(kwargs)

            start = time.perf_counter()

            with mlflow.start_span(name=span_name, span_type=span_type) as span:
                # Set inputs
                if inputs:
```

```
                span.set_inputs(inputs)

            # Static attributes
            for key, value in static_attrs.items():
                span.set_attribute(key, value)

            # Function metadata
            span.set_attribute("func.name", func.__name__)
            span.set_attribute("func.module", func.__module__)

            # Input size tracking
            span.set_attribute("bytes.input", input_size)

            # Estimate input tokens if model specified
            if model:
                input_text = " ".join(input_text_parts)
                input_tokens = estimate_tokens(input_text)
                span.set_attribute("tokens.input", input_tokens)
                span.set_attribute("model.name", model)

            try:
                result = func(*args, **kwargs)

                # Timing
                elapsed = round((time.perf_counter() - start) * 1000, 2)
                span.set_attribute("duration_ms", elapsed)

                # Output size
                output_size = calculate_size(result)
                span.set_attribute("bytes.output", output_size)

                # Estimate output tokens and calculate cost
                if model and isinstance(result, dict):
                    output_text = json.dumps(result) if result else ""
                    output_tokens = estimate_tokens(output_text)
                    span.set_attribute("tokens.output", output_tokens)

                    total_tokens = span.get_attribute("tokens.input") + outp
                    cost = calculate_cost(model, total_tokens, 0)  # Simpli
                    span.set_attribute("cost.usd", cost)
                    span.set_attribute("tokens.total", total_tokens)

                # Smart output extraction
                if isinstance(result, dict):
                    outputs = {
                        "response": result  # Explicitly capture full respor
                    }
                    if "error" in result:
                        outputs["error"] = result["error"]
                        span.set_attribute("error", True)
                    if "rows" in result:
                        outputs["row_count"] = len(result["rows"])
                        span.set_attribute("db.rows_returned", len(result["r
```

```
            if "sql" in result:
                outputs["sql"] = result["sql"]  # Capture full SQL
            if "table_md" in result:
                outputs["table_md"] = result["table_md"]  # Capture
            if "chunks" in result:
                outputs["chunk_count"] = len(result["chunks"])
                outputs["chunks"] = result["chunks"]  # Capture full
                span.set_attribute("retrieval.chunks", len(result["c
            if "sources" in result:
                outputs["sources"] = result["sources"]  # Capture so
                span.set_attribute("retrieval.sources", len(result['
            if "results" in result:
                outputs["result_count"] = len(result["results"])
                # Extract top-k links for web search results
                if result["results"] and isinstance(result["results'
                    top_links = [item.get("url") for item in result[
                    if top_links:
                        outputs["top_links"] = top_links
                        span.set_attribute("search.top_links_count",

            span.set_outputs(outputs)
        else:
            # For non-dict results, still capture as response
            span.set_outputs({"response": result})

        # Update trace-level request and response
        # Extract request from first string argument (user_question,
        request_str = None
        for i, arg_name in enumerate(arg_names):
            if i < len(args):
                val = args[i]
            elif arg_name in kwargs:
                val = kwargs[arg_name]
            else:
                continue

            # If it's a string and looks like a query/message argume
            if isinstance(val, str) and any(keyword in arg_name.lowe
                request_str = val
                break

        # Fallback: use first string argument if no obvious request
        if not request_str:
            for i, arg_name in enumerate(arg_names):
                if i < len(args):
                    val = args[i]
                elif arg_name in kwargs:
                    val = kwargs[arg_name]
                else:
                    continue
                if isinstance(val, str):
                    request_str = val
                    break
```

```
                    response_str = None
                    if isinstance(result, dict):
                        response_str = json.dumps(result)
                    elif isinstance(result, str):
                        response_str = result
                    else:
                        response_str = str(result)

                    # Update the trace with request preview and response preview
                    # Only include parameters that are not None to avoid nullify
                    trace_update_kwargs = {}
                    if request_str:
                        trace_update_kwargs["request_preview"] = request_str[:50
                    if response_str:
                        trace_update_kwargs["response_preview"] = response_str[:

                    if trace_update_kwargs:
                        mlflow.update_current_trace(**trace_update_kwargs)

                    return result

                except Exception as e:
                    span.set_attribute("error", True)
                    span.set_attribute("error.message", str(e))
                    span.set_attribute("error.type", type(e).__name__)
                    raise

        return wrapper
    return decorator

# ── Context Manager (Also with cost/size tracking) ──────────────────────

@contextmanager
def trace_span(name, span_type="CHAIN", attributes=None, model=None):
    """Trace a block of code with full tracking."""
    if not ENABLED:
        yield None
        return

    _init()

    with mlflow.start_span(name=name, span_type=span_type) as span:
        start = time.perf_counter()

        if attributes:
            for key, value in attributes.items():
                span.set_attribute(key, value)

        if model:
            span.set_attribute("model.name", model)

        try:
```

```
            yield span

            elapsed = round((time.perf_counter() - start) * 1000, 2)
            span.set_attribute("duration_ms", elapsed)

        except Exception as e:
            span.set_attribute("error", True)
            span.set_attribute("error.message", str(e))
            raise

# ── Helper: Manual Attribute Updates ──────────────────────────────

def set_attr(key: str, value):
    """Set attribute on current span."""
    if ENABLED:
        current = mlflow.get_current_active_span()
        if current:
            current.set_attribute(key, value)

def set_attrs(attributes: dict):
    """Set multiple attributes."""
    if ENABLED:
        current = mlflow.get_current_active_span()
        if current:
            for key, value in attributes.items():
                current.set_attribute(key, value)

def log_cost(model: str, input_tokens: int, output_tokens: int):
    """Manually log cost for a span."""
    if ENABLED:
        cost = calculate_cost(model, input_tokens, output_tokens)
        set_attrs({
            "cost.usd": cost,
            "tokens.input": input_tokens,
            "tokens.output": output_tokens,
            "tokens.total": input_tokens + output_tokens,
            "model.name": model
        })
```

6. Defining the Tool

6.1 RAG Tool

- Ingests PDF documents, splits them into overlapping chunks, and converts them into embeddings using a local sentence-transformer model
- Stores embeddings in a FAISS vector index for efficient semantic similarity search

- At query time, retrieves the most relevant chunks based on the user question to provide grounded context
- Returns retrieved text and source metadata, while leaving final answer generation to downstream LLM components

```
# tools/rag_tool.py
# ─────────────────────────────────────────────────────────────────────────────
# RAG Tool: PDF ingestion → FAISS vector store → chunk retrieval
#
# Embedding model: sentence-transformers/all-MiniLM-L6-v2  (runs locally, free)
# Vector store:    FAISS (CPU)
#
# NOTE: This tool only retrieves relevant chunks. Answer synthesis
# is handled by the central synthesis node.
#
# Usage:
#   # One-time build (or whenever PDFs change):
#   python -c "from tools.rag_tool import build_index; build_index()"
#
#   # Query:
#   result = run_rag_tool("What is the return policy?", history)
# ─────────────────────────────────────────────────────────────────────────────

import os
import json
import pickle
import re
import numpy as np
import faiss
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from observability import trace  # NEW: Import observability


from config import (
    PDF_DIR, FAISS_INDEX_DIR, EMBEDDING_MODEL,
    CHUNK_SIZE, CHUNK_OVERLAP, RAG_TOP_K,
)

# ── Embedding helper ──────────────────────────────────────────────────────────

_embedder = None  # Global embedder cache

def _get_embedder() -> SentenceTransformer:
    global _embedder
    if _embedder is None:
        print(f"[rag_tool] Loading embedding model: {EMBEDDING_MODEL} …")
        _embedder = SentenceTransformer(EMBEDDING_MODEL)
    return _embedder
```

```
def embed_texts(texts: list[str]) -> np.ndarray:
    """Return (N, D) float32 array of L2-normalised embeddings."""
    model  = _get_embedder()
    vecs   = model.encode(texts, show_progress_bar=False, normalize_embeddings=T
    return np.array(vecs, dtype="float32")

# ── PDF → chunks ──────────────────────────────────────────────────────────

def _extract_text_from_pdf(pdf_path: str) -> str:
    """Extract all text from a PDF file."""
    reader = PdfReader(pdf_path)
    pages  = []
    for page in reader.pages:
        text = page.extract_text() or ""
        pages.append(text)
    return "\n".join(pages)


def _chunk_text(text: str, source: str) -> list[dict]:
    """
    Split text into overlapping chunks.
    Each chunk is a dict: {"text": str, "source": str, "chunk_id": int}
    """
    text   = re.sub(r"\s+", " ", text).strip()
    chunks = []
    start  = 0
    idx    = 0
    while start < len(text):
        end   = min(start + CHUNK_SIZE, len(text))
        chunk = text[start:end].strip()
        if chunk:
            chunks.append({"text": chunk, "source": source, "chunk_id": idx})
            idx += 1
        start += CHUNK_SIZE - CHUNK_OVERLAP
    return chunks

# ── Index build & load ────────────────────────────────────────────────────

_INDEX_FILE = None   # will be set after FAISS_INDEX_DIR is resolved
_META_FILE  = None


def _index_paths():
    os.makedirs(FAISS_INDEX_DIR, exist_ok=True)
    idx_file  = os.path.join(FAISS_INDEX_DIR, "index.faiss")
    meta_file = os.path.join(FAISS_INDEX_DIR, "metadata.pkl")
    return idx_file, meta_file


def build_index(force: bool = False) -> None:
    """
```

```
    Ingest all PDFs in PDF_DIR and build a FAISS flat IP index.
    Saves index + metadata to FAISS_INDEX_DIR.
    """
    idx_file, meta_file = _index_paths()

    if os.path.exists(idx_file) and not force:
        print(f"[rag_tool] FAISS index already exists at {idx_file}. Skipping bu
        print("           Call build_index(force=True) to rebuild.")
        return

    pdf_files = [
        os.path.join(PDF_DIR, f)
        for f in os.listdir(PDF_DIR)
        if f.lower().endswith(".pdf")
    ]

    if not pdf_files:
        print(f"[rag_tool] WARNING: No PDFs found in {PDF_DIR}.")
        print("           Add PDF files there and re-run build_index().")
        # Build an empty (but valid) index so the rest of the code doesn't crash
        dim   = 384   # all-MiniLM-L6-v2 output dimension
        index = faiss.IndexFlatIP(dim)
        faiss.write_index(index, idx_file)
        with open(meta_file, "wb") as fh:
            pickle.dump([], fh)
        return

    all_chunks: list[dict] = []
    for pdf_path in pdf_files:
        source = os.path.basename(pdf_path)
        print(f"[rag_tool] Parsing {source} …")
        text   = _extract_text_from_pdf(pdf_path)
        chunks = _chunk_text(text, source)
        all_chunks.extend(chunks)
        print(f"           → {len(chunks)} chunks")

    texts  = [c["text"] for c in all_chunks]
    vecs   = embed_texts(texts)

    dim    = vecs.shape[1]
    index  = faiss.IndexFlatIP(dim)   # Inner Product on normalised vecs = cosir
    index.add(vecs)

    faiss.write_index(index, idx_file)
    with open(meta_file, "wb") as fh:
        pickle.dump(all_chunks, fh)

    print(f"[rag_tool] ✓ Index built: {len(all_chunks)} chunks, dim={dim}")


def _load_index():
    """Load FAISS index and metadata from disk. Build if missing."""
    idx_file, meta_file = _index_paths()
```

```
    if not os.path.exists(idx_file):
        build_index()
    index = faiss.read_index(idx_file)
    with open(meta_file, "rb") as fh:
        metadata = pickle.load(fh)
    return index, metadata


# ── Retrieval ──────────────────────────────────────────────────────────

def retrieve_chunks(query: str, k: int = RAG_TOP_K) -> list[dict]:
    """
    Embed query and return top-k most similar chunks.
    Each returned dict: {"text": str, "source": str, "chunk_id": int, "score": f
    """
    index, metadata = _load_index()

    if index.ntotal == 0:
        return []

    q_vec  = embed_texts([query])
    scores, idxs = index.search(q_vec, min(k, index.ntotal))

    results = []
    for score, i in zip(scores[0], idxs[0]):
        if i < 0:
            continue
        chunk = dict(metadata[i])
        chunk["score"] = float(score)
        results.append(chunk)
    return results

@trace(span_type="RETRIEVER", attributes={  # NEW: Add tracing
    "retrieval.model": EMBEDDING_MODEL,
    "retrieval.top_k": RAG_TOP_K,
})
def run_rag_tool(user_question: str, conversation_history: list[dict]) -> dict:
    """
    Main entry point for the RAG tool.

    Returns a dict:
      {
        "chunks":  list[dict],   # retrieved chunks with scores and text
        "sources": list[str],    # unique source document names
      }
    """
    chunks = retrieve_chunks(user_question, k=RAG_TOP_K)

    sources = list({c["source"] for c in chunks})

    return {
        "chunks":  chunks,
```

```
        "sources": sources,
    }
```

6.2 SQL Tool (Text2SQL Execution Engine)

- Translates natural language questions into SQLite SQL queries using an LLM, guided by a detailed schema and join relationships
- Executes the generated SQL against a local database and returns structured results
- Formats query outputs into a readable markdown table for downstream consumption
- Handles errors gracefully and exposes both the generated SQL and execution results for observability and debugging.

```
# tools/sql_tool.py
# ─────────────────────────────────────────────────────────────────────────────
# SQL Tool: generates and executes SQL queries against the local SQLite DB.
#
# Flow:
#   1. Build a system prompt that includes all table schemas + sample rows
#   2. Call GPT-4o-mini to translate the user question into SQLite SQL
#   3. Execute the SQL; return results as a markdown table string
# ─────────────────────────────────────────────────────────────────────────────

import sqlite3
import json
from openai import OpenAI
from config import DB_PATH, LLM_MODEL, LLM_TEMPERATURE, LLM_MAX_TOKENS, OPENAI_A
from observability import trace  # NEW: Import observability

client = OpenAI(api_key=OPENAI_API_KEY)

# ── Schema documentation (hand-written, mirrors Kaggle dataset) ──────────────
SCHEMA_DESCRIPTION = """
You have access to a SQLite database with the following tables:

TABLE: orders
  order_id                      TEXT  (primary key)
  customer_id                   TEXT
  order_status                  TEXT  (delivered, cancelled, invoiced, processir
  order_purchase_timestamp      TEXT  (ISO datetime)
  order_approved_at             TEXT  (ISO datetime)
  order_delivered_timestamp     TEXT  (ISO datetime, may be NULL)
  order_estimated_delivery_date TEXT  (ISO date)
```

```
TABLE: order_items
  order_id          TEXT
  order_item_id     INTEGER  (item sequence within an order)
  product_id        TEXT
  seller_id         TEXT
  price             REAL
  shipping_charges  REAL

TABLE: customers
  customer_id             TEXT  (primary key)
  customer_zip_code_prefix TEXT
  customer_city           TEXT
  customer_state          TEXT

TABLE: payments
  order_id              TEXT
  payment_sequential    INTEGER
  payment_type          TEXT  (credit_card, boleto, voucher, debit_card)
  payment_installments  INTEGER
  payment_value         REAL

TABLE: products
  product_id            TEXT  (primary key)
  product_category_name TEXT
  product_weight_g      REAL
  product_length_cm     REAL
  product_height_cm     REAL
  product_width_cm      REAL

JOIN HINTS:
  orders.customer_id       → customers.customer_id
  order_items.order_id     → orders.order_id
  order_items.product_id   → products.product_id
  payments.order_id        → orders.order_id

RULES:
  - Output ONLY the raw SQLite SQL query, no markdown, no explanation.
  - Use table aliases for readability.
  - For date filtering use: strftime('%Y-%m', order_purchase_timestamp) = '2018-
  - Always add LIMIT 50 unless the user asks for all rows.
  - Aggregate with SUM / COUNT / AVG / GROUP BY as appropriate.
  - Never use SELECT *; always name columns explicitly.
"""

SQL_SYSTEM_PROMPT = (
    "You are an expert SQLite query writer for an e-commerce analytics database.
    + SCHEMA_DESCRIPTION
)


def _generate_sql(user_question: str, conversation_history: list[dict]) -> str:
    """Call GPT-4o-mini to translate a natural language question into SQLite SQL
```

```
    messages = [{"role": "system", "content": SQL_SYSTEM_PROMPT}]

    # Include recent conversation for multi-turn context (last 6 messages = 3 tu
    messages.extend(conversation_history[-6:])
    messages.append({"role": "user", "content": user_question})

    response = client.chat.completions.create(
        model=LLM_MODEL,
        messages=messages,
        temperature=LLM_TEMPERATURE,
        max_tokens=512,
    )
    return response.choices[0].message.content.strip()


def _execute_sql(sql: str) -> tuple[list[dict], str | None]:
    """
    Execute sql against the SQLite DB.
    Returns (rows_as_list_of_dicts, error_string_or_None).
    """
    try:
        conn = sqlite3.connect(DB_PATH)
        conn.row_factory = sqlite3.Row
        cur  = conn.cursor()
        cur.execute(sql)
        rows = [dict(r) for r in cur.fetchall()]
        conn.close()
        return rows, None
    except sqlite3.Error as exc:
        return [], str(exc)


def _rows_to_markdown(rows: list[dict], max_rows: int = 20) -> str:
    """Convert a list of dicts into a compact markdown table string."""
    if not rows:
        return "_No rows returned._"

    headers  = list(rows[0].keys())
    display  = rows[:max_rows]
    header_line = "| " + " | ".join(headers) + " |"
    sep_line    = "| " + " | ".join(["---"] * len(headers)) + " |"
    data_lines  = [
        "| " + " | ".join(str(r.get(h, "")) for h in headers) + " |"
        for r in display
    ]
    table = "\n".join([header_line, sep_line] + data_lines)

    if len(rows) > max_rows:
        table += f"\n\n_… and {len(rows) - max_rows} more rows (showing first {r
    return table

@trace(span_type="TOOL", model="gpt-4o-mini", attributes={  # NEW: Add tracing
    "db.type": "sqlite",
```

```
    "db.path": DB_PATH,
})
def run_sql_tool(user_question: str, conversation_history: list[dict]) -> dict:
    """
    Main entry point for the SQL tool.

    Returns a dict:
      {
        "sql":      str,          # the generated SQL
        "rows":     list[dict],   # raw query results
        "table_md": str,          # markdown-formatted result table
        "error":    str | None,   # execution error, if any
      }
    """
    sql   = _generate_sql(user_question, conversation_history)
    rows, error = _execute_sql(sql)

    return {
        "sql":      sql,
        "rows":     rows,
        "table_md": _rows_to_markdown(rows) if not error else f"_SQL error: {err
        "error":    error,
    }
```

6.3 Websearch Tool

- Fetches real-time information from the web using the Serper API, returning structured results (title, URL, snippet)
- Acts as a fallback for queries that cannot be answered using internal data or RAG context
- Returns clean, LLM-friendly search results for downstream synthesis and reasoning

```
# tools/web_search_tool.py
# ─────────────────────────────────────────────────────────────────────────────
# Web Search Tool: uses Serper API to fetch live web results.
#
# Serper returns structured results (title, link, snippet) — much
# cleaner than raw HTML scraping for LLM consumption.
#
# NOTE: This tool only retrieves raw search results. Answer synthesis
# is handled by the central synthesis node.
# ─────────────────────────────────────────────────────────────────────────────

import requests
```

```
from config import SERPER_API_KEY
from observability import trace  # NEW: Import observability

@trace(span_type="TOOL", attributes={  # NEW: Add tracing
    "api.provider": "serper.dev",
    "api.endpoint": "google.serper.dev/search",
})
def run_web_search_tool(user_question: str, conversation_history: list[dict]) ->
    """
    Main entry point for the Web Search tool.

    Steps:
      1. Call Serper API to search the web
      2. Return raw results for synthesis node to handle

    Returns a dict:
      {
        "results": list[dict],   # raw Serper results [{title, url, content}]
        "query":   str,          # the query sent to Serper
      }
    """
    # ── 1. Web search ──────────────────────────────────────────────────────
    try:
        if not SERPER_API_KEY:
            return {
                "results": [],
                "query": user_question,
            }

        url = "https://google.serper.dev/search"
        headers = {
            "X-API-KEY": SERPER_API_KEY,
            "Content-Type": "application/json"
        }
        payload = {"q": user_question, "num": 5}

        response = requests.post(url, json=payload, headers=headers, timeout=10)

        if response.status_code != 200:
            return {
                "results": [],
                "query": user_question,
            }

        data = response.json()
        raw_results = []

        # Extract organic results
        for item in data.get("organic", [])[:5]:
            raw_results.append({
                "title": item.get("title"),
                "url": item.get("link"),
                "content": item.get("snippet")
```

```
            })
    except Exception as exc:
        print(f"[web_search_tool] Error: {exc}")
        return {
            "results": [],
            "query": user_question,
        }

    return {
        "results": raw_results,
        "query": user_question,
    }
```

7. Defining the Agentic Flow

We will use `LangGraph` to define the agentic flow.

7.1 Defining the Agent State

We now defines the shared LangGraph state object that carries user input, routing decisions, tool outputs, and the final response across all nodes in the pipeline.

```
# agent/state.py
# ─────────────────────────────────────────────────────────────────────
# LangGraph state definition.
# All fields flow through the graph; nodes read what they need and return
# only the keys they update.
# ─────────────────────────────────────────────────────────────────────

from typing import TypedDict, Optional, Any


class AgentState(TypedDict):
    # ── Input ──────────────────────────────────────────────────────────
    user_message: str
    conversation_history: list[dict]   # [{"role": "user"|"assistant", "content"

    # ── Routing ────────────────────────────────────────────────────────
    route:        str                  # "sql" | "rag" | "web_search"
    route_reason: str                  # explanation from the router LLM

    # ── Tool outputs (only one will be populated per turn) ─────────────
    sql_result:        Optional[dict]  # {sql, rows, table_md, error}
    rag_result:        Optional[dict]  # {answer, chunks, sources}
    web_search_result: Optional[dict]  # {answer, results, query}

    # ── Final synthesised answer ───────────────────────────────────────
    final_answer: str
```

```
    # ── Turn metadata ──────────────────────────────────────────────────
    turn_number:  int
```

7.2 Router

It routes each user query to the appropriate path (SQL, RAG, or web search) using an LLM-based classifier with reasoning for the decision.

Notice that we have added a decorator `@trace(span_type="PARSER", model="gpt-4o-mini")` and span during the function `span = mlflow.get_current_active_span()` for logging.

```
# agent/router.py
# ──────────────────────────────────────────────────────────────────────
# Router: uses GPT-4o-mini to classify every user message into one of three
# routes before handing off to the appropriate tool node.
#
# Routes:
#   sql        → query the local SQLite e-commerce database
#   rag        → search loaded PDF documents
#   web_search → live internet search via Tavily
# ──────────────────────────────────────────────────────────────────────

import json
import re
```



```
client = OpenAI(api_key=OPENAI_API_KEY)

ROUTER_SYSTEM = """\
You are a routing agent for an e-commerce analytics assistant.
Given a user message and conversation history, you must decide which tool to use

AVAILABLE TOOLS:
  sql        — Use when the question asks about data that lives in the database:
               orders, order items, customers, payments, products, revenue,
               counts, aggregations, trends, specific orders/customers, SQL-styl
               queries over structured e-commerce data.

  rag        — Use when the question asks about information in uploaded document
               policies, manuals, documentation, FAQs, anything the user has
               uploaded as a PDF. If no PDFs are loaded this still applies.

  web_search — Use when the question:
```

```
                • asks about current events, news, or real-time information
                • is about general e-commerce industry trends or benchmarks
                • cannot be answered from the database or PDF documents
                • asks "what is", "how does X work", or general knowledge questic

OUTPUT FORMAT (respond with ONLY valid JSON, no other text):
{
  "route": "<sql|rag|web_search>",
  "reason": "<one-sentence explanation>"
}
"""

@trace(span_type="PARSER", model="gpt-4o-mini")
def route_question(user_message: str, conversation_history: list[dict]) -> dict:
    """
    Classify user_message into one of ROUTES.

    Returns:
      {
        "route":  str,   # one of "sql", "rag", "web_search"
        "reason": str,   # routing rationale
      }
    """
    messages = [{"role": "system", "content": ROUTER_SYSTEM}]

    # Add brief conversation context (last 4 messages = 2 turns)
    for msg in conversation_history[-4:]:
        messages.append({"role": msg["role"], "content": msg["content"]})

    messages.append({"role": "user", "content": user_message})

    # Capture system and user prompts as span inputs
    span = mlflow.get_current_active_span()
    if span:
        span.set_inputs({
            "system_prompt": ROUTER_SYSTEM,
            "user_prompt": user_message,
            "conversation_context": conversation_history[-4:]
        })

    response = client.chat.completions.create(
        model=LLM_MODEL,
        messages=messages,
        temperature=0.0,
        max_tokens=128,
        response_format={"type": "json_object"},
    )

    raw = response.choices[0].message.content.strip()

    try:
        parsed = json.loads(raw)
    except json.JSONDecodeError:
```

```
            # Fallback: extract route from raw text
            for route in ROUTES:
                if route in raw.lower():
                    return {"route": route, "reason": "extracted from malformed JSON
            return {"route": "web_search", "reason": "could not parse router output"

        route = parsed.get("route", "web_search").lower().strip()
        if route not in ROUTES:
            route = "web_search"

        return {
            "route":  route,
            "reason": parsed.get("reason", ""),
        }
```

7.3 LangGraph Nodes

The `nodes.py` module defines the core execution logic of the pipeline, where each node performs a specific step: routing, tool invocation, or response synthesis, while passing state through the graph.

Together, these nodes form a modular, observable workflow that enables dynamic decision-making across SQL, RAG, and web search paths.

Here:

- router_node() calls `route_question()` to choose `sql`, `rag`, or `web_search`.

- `sql_node()` calls `run_sql_tool()`.

- `rag_node()` calls `run_rag_tool()`.

- `web_search_node()` calls `run_web_search_tool()`.

- `synthesise_node()` builds a final answer with the LLM.

- `update_history_node()` appends the turn to history.

```
# agent/nodes.py
# ─────────────────────────────────────────────────────────────────────────
# LangGraph node functions.  Each node receives the full AgentState dict and
# returns a partial dict with only the keys it updates.
# ─────────────────────────────────────────────────────────────────────────
```

```
import mlflow
from openai import OpenAI
from agent.router import route_question
from tools.sql_tool        import run_sql_tool
from tools.rag_tool        import run_rag_tool
from tools.web_search_tool import run_web_search_tool
from config import LLM_MODEL, LLM_TEMPERATURE, LLM_MAX_TOKENS, OPENAI_API_KEY
from observability import trace, set_attrs  # Import observability utilities

client = OpenAI(api_key=OPENAI_API_KEY)


# ─────────────────────────────────────────────────────────────────────────────
# Node 1 — Router
# ─────────────────────────────────────────────────────────────────────────────

def router_node(state: dict) -> dict:
    """
    Classify the user message and set state["route"].
    """
    routing = route_question(
        user_message=state["user_message"],
        conversation_history=state["conversation_history"],
    )
    print(f"\n[router] → {routing['route'].upper()}  |  {routing['reason']}")
    return {
        "route":        routing["route"],
        "route_reason": routing["reason"],
    }


# ─────────────────────────────────────────────────────────────────────────────
# Node 2a — SQL Tool Node
# ─────────────────────────────────────────────────────────────────────────────

def sql_node(state: dict) -> dict:
    """
    Generate & execute a SQL query, store raw result in state.
    """
    print("[sql_node] Generating and executing SQL …")
    result = run_sql_tool(
        user_question=state["user_message"],
        conversation_history=state["conversation_history"],
    )
    print(f"[sql_node] SQL: {result['sql']}")
    if result["error"]:
        print(f"[sql_node] ERROR: {result['error']}")
    else:
        print(f"[sql_node] {len(result['rows'])} rows returned.")
    return {"sql_result": result}


# ─────────────────────────────────────────────────────────────────────────────
```

```
# Node 2b — RAG Tool Node
# ──────────────────────────────────────────────────────────────────────

def rag_node(state: dict) -> dict:
    """
    Retrieve relevant PDF chunks and synthesise an answer.
    """
    print("[rag_node] Retrieving document chunks …")
    result = run_rag_tool(
        user_question=state["user_message"],
        conversation_history=state["conversation_history"],
    )
    print(f"[rag_node] {len(result['chunks'])} chunks retrieved from: {result['s
    return {"rag_result": result}


# ──────────────────────────────────────────────────────────────────────
# Node 2c — Web Search Tool Node
# ──────────────────────────────────────────────────────────────────────

def web_search_node(state: dict) -> dict:
    """
    Run Tavily search and synthesise an answer.
    """
    print("[web_node] Searching the web …")
    result = run_web_search_tool(
        user_question=state["user_message"],
        conversation_history=state["conversation_history"],
    )
    print(f"[web_node] {len(result['results'])} results for query: '{result['que
    return {"web_search_result": result}


# ──────────────────────────────────────────────────────────────────────
# Node 3 — Answer Synthesiser
# ──────────────────────────────────────────────────────────────────────

_SYNTHESISE_SYSTEM = """\
You are a friendly and precise e-commerce analytics assistant.
Your job is to deliver a clear, well-formatted final answer to the user.

Guidelines:
- Be concise but complete.
- If data comes from SQL results, summarise the key numbers first, then explain.
- If data comes from documents, cite the source.
- If data comes from web search, mention the URLs.
- Use markdown formatting where it aids readability (bullet lists, bold, tables)
- Maintain a helpful, professional tone.
"""

@trace(span_type="CHAIN", model="gpt-4o-mini")  # NEW: Add tracing
def synthesise_node(state: dict) -> dict:
    """
```

```
    Combine tool output into a polished final answer using GPT-4o-mini.
    - For SQL: synthesize narrative from raw rows
    - For RAG: synthesize answer from retrieved document chunks
    - For Web Search: synthesize answer from web search results
    """
    route = state.get("route", "")

    # ── SQL: tool returns raw rows — synthesise a narrative ──────────────
    if route == "sql":
        sql_result = state.get("sql_result", {})
        sql_query  = sql_result.get("sql", "")
        table_md   = sql_result.get("table_md", "_No results_")
        error      = sql_result.get("error")

        if error:
            content = (
                f"The SQL query encountered an error:\n\n"
                f"```sql\n{sql_query}\n```\n\n"
                f"Error: {error}\n\n"
                "Please try rephrasing your question."
            )
        else:
            content = (
                f"SQL query executed:\n```sql\n{sql_query}\n```\n\n"
                f"Results:\n{table_md}"
            )

        messages = [{"role": "system", "content": _SYNTHESISE_SYSTEM}]
        messages.extend(state["conversation_history"][-4:])
        user_content = (
            f"Original question: {state['user_message']}\n\n"
            f"Query output:\n{content}\n\n"
            "Provide a clear, business-friendly summary of these results."
        )
        messages.append({"role": "user", "content": user_content})

        # Capture prompts as span inputs
        span = mlflow.get_current_active_span()
        if span:
            span.set_inputs({
                "system_prompt": _SYNTHESISE_SYSTEM,
                "user_prompt": user_content,
                "synthesis_type": "sql",
                "user_message": state['user_message']
            })

        response = client.chat.completions.create(
            model=LLM_MODEL,
            messages=messages,
            temperature=LLM_TEMPERATURE,
            max_tokens=LLM_MAX_TOKENS,
        )
        answer = response.choices[0].message.content.strip()
```

```
    # ── RAG: synthesize answer from retrieved document chunks ──────────────
    elif route == "rag":
        rag_result = state.get("rag_result", {})
        chunks     = rag_result.get("chunks", [])
        sources    = rag_result.get("sources", [])

        if not chunks:
            answer = "I could not find relevant information in the loaded docume
        else:
            # Build context block from retrieved chunks
            context_parts = []
            for i, chunk in enumerate(chunks, 1):
                context_parts.append(
                    f"[Excerpt {i} — {chunk['source']} (score: {chunk['score']:.
                    f"{chunk['text']}"
                )
            context = "\n\n---\n\n".join(context_parts)

            messages = [{"role": "system", "content": _SYNTHESISE_SYSTEM}]
            messages.extend(state["conversation_history"][-4:])
            user_content = (
                f"Document excerpts:\n\n{context}\n\n"
                f"Question: {state['user_message']}"
            )
            messages.append({"role": "user", "content": user_content})

            # Capture prompts as span inputs
            span = mlflow.get_current_active_span()
            if span:
                span.set_inputs({
                    "system_prompt": _SYNTHESISE_SYSTEM,
                    "user_prompt": user_content,
                    "synthesis_type": "rag",
                    "retrieval_sources": ", ".join(sources) if sources else "nor
                    "user_message": state['user_message']
                })

            response = client.chat.completions.create(
                model=LLM_MODEL,
                messages=messages,
                temperature=LLM_TEMPERATURE,
                max_tokens=LLM_MAX_TOKENS,
            )
            answer = response.choices[0].message.content.strip()

            if sources:
                answer += f"\n\n_Sources: {', '.join(sources)}_"

    # ── Web Search: synthesize answer from search results ──────────────────
    elif route == "web_search":
        web_result = state.get("web_search_result", {})
        results    = web_result.get("results", [])
```

```
            query      = web_result.get("query", "")

            if not results:
                answer = "The web search returned no results for your query."
            else:
                # Format results for context
                result_blocks = []
                for i, r in enumerate(results, 1):
                    block = (
                        f"[Result {i}]\n"
                        f"Title:   {r.get('title', 'N/A')}\n"
                        f"URL:     {r.get('url', 'N/A')}\n"
                        f"Snippet: {r.get('content', '')}"
                    )
                    result_blocks.append(block)
                formatted_results = "\n\n".join(result_blocks)

                messages = [{"role": "system", "content": _SYNTHESISE_SYSTEM}]
                messages.extend(state["conversation_history"][-4:])
                user_content = (
                    f"Web search results for '{query}':\n\n{formatted_results}\n\n"
                    f"Original question: {state['user_message']}\n\n"
                    "Synthesize a clear, accurate answer from these results. "
                    "Always mention the source URLs so the user can verify."
                )
                messages.append({"role": "user", "content": user_content})

                # Capture prompts as span inputs
                span = mlflow.get_current_active_span()
                if span:
                    span.set_inputs({
                        "system_prompt": _SYNTHESISE_SYSTEM,
                        "user_prompt": user_content,
                        "synthesis_type": "web_search",
                        "web_search_query": query,
                        "user_message": state['user_message']
                    })

                response = client.chat.completions.create(
                    model=LLM_MODEL,
                    messages=messages,
                    temperature=LLM_TEMPERATURE,
                    max_tokens=LLM_MAX_TOKENS,
                )
                answer = response.choices[0].message.content.strip()

        else:
            answer = "I was unable to determine how to answer your question."

        print(f"[synthesise_node] Answer ready ({len(answer)} chars).")
        return {"final_answer": answer}
```

```
# ──────────────────────────────────────────────────────────────
# Node 4 — History Updater
# ──────────────────────────────────────────────────────────────

def update_history_node(state: dict) -> dict:
    """
    Append the current turn to conversation_history.
    (Turn number is already incremented in session.ask())
    """
    history = list(state["conversation_history"])
    history.append({"role": "user",      "content": state["user_message"]})
    history.append({"role": "assistant", "content": state["final_answer"]})
    return {
        "conversation_history": history,
        "turn_number": state.get("turn_number", 0),
    }
```

7.4 LangGraph Graph

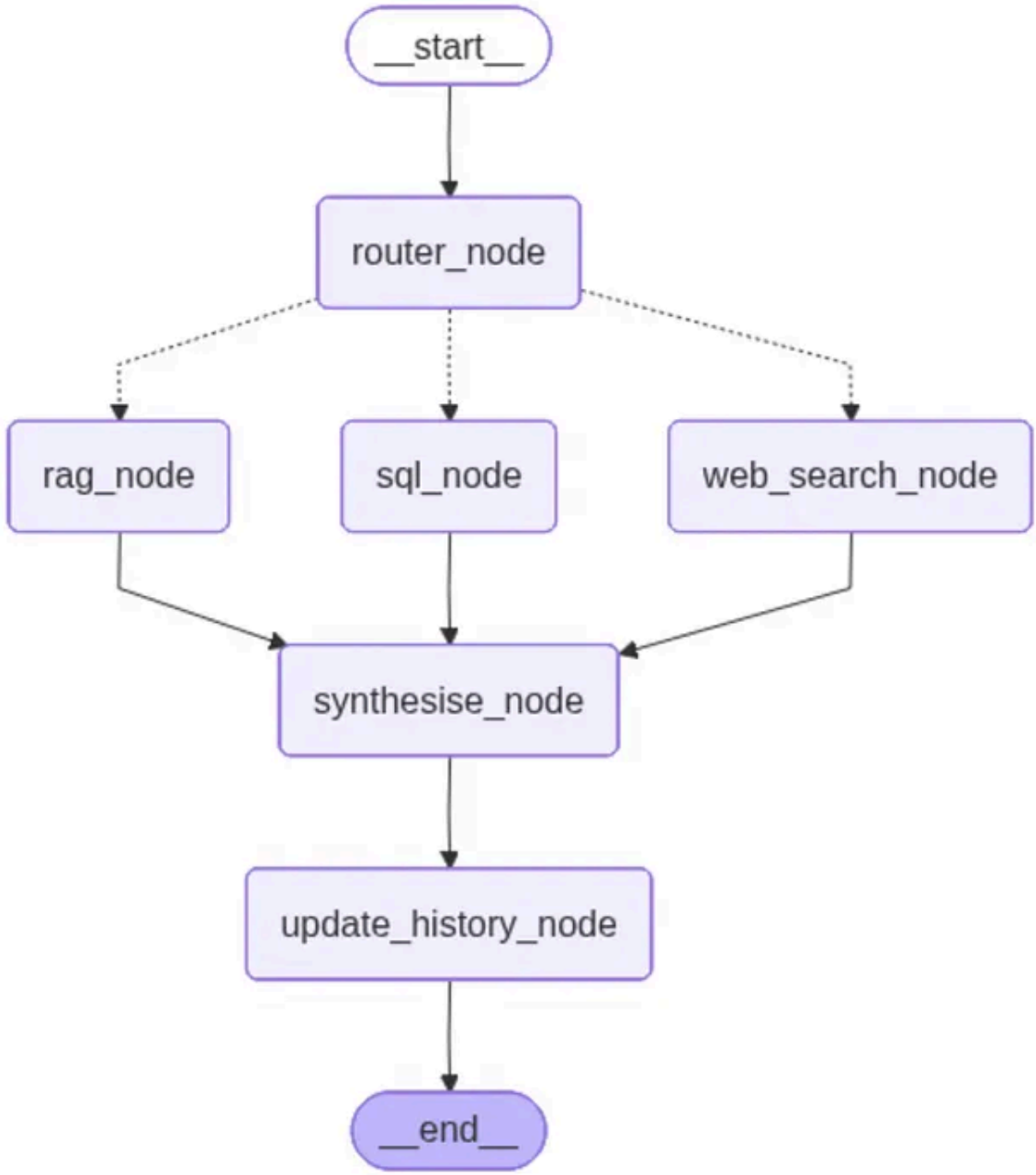
The `graph.py` module defines the overall LangGraph workflow, connecting all nodes into a conditional execution graph that dynamically routes user queries through SQL, RAG, or web search paths before synthesizing a final response.

- Implements conditional routing logic to direct queries to the correct tool node based on the router's decision
- Orchestrates the full pipeline flow from `routing → tool execution → synthesis → conversation history update`

```
# agent/graph.py
# ──────────────────────────────────────────────────────────────
# LangGraph graph definition.
#
# Graph topology:
#
#   [START]
#     │
#     ▼
#   router_node
#     │
#     ├── "sql"        ──▶ sql_node        ──┐
#     ├── "rag"        ──▶ rag_node        ──┤
#     └── "web_search" ──▶ web_search_node ──┤
#                                            │
#                                            ▼
#                                     synthesise_node
#                                            │
```

```
#                                        ▼
#                                 update_history_node
#                                        |
#                                      [END]
# ──────────────────────────────────────────────────────────────────────────

from langgraph.graph import StateGraph, END
from agent.state import AgentState
from agent.nodes import (
    router_node,
    sql_node,
    rag_node,
    web_search_node,
    synthesise_node,
    update_history_node,
)


def _route_selector(state: dict) -> str:
    """Conditional edge: read state["route"] and return the target node name."""
    route = state.get("route", "web_search")
    return {
        "sql":        "sql_node",
        "rag":        "rag_node",
        "web_search": "web_search_node",
    }.get(route, "web_search_node")


def build_graph() -> StateGraph:
    """
    Assemble and compile the LangGraph routing agent.
    Returns a compiled graph ready to invoke.
    """
    graph = StateGraph(AgentState)

    # ── Register nodes ──────────────────────────────────────────────────
    graph.add_node("router_node",       router_node)
    graph.add_node("sql_node",          sql_node)
    graph.add_node("rag_node",          rag_node)
    graph.add_node("web_search_node",   web_search_node)
    graph.add_node("synthesise_node",   synthesise_node)
    graph.add_node("update_history_node", update_history_node)

    # ── Entry point ─────────────────────────────────────────────────────
    graph.set_entry_point("router_node")

    # ── Conditional routing: router → one of the three tool nodes ───────
    graph.add_conditional_edges(
        source="router_node",
        path=_route_selector,
        path_map={
            "sql_node":        "sql_node",
            "rag_node":        "rag_node",
```

```
            "web_search_node": "web_search_node",
        },
    )

    # ── All tool nodes feed into synthesis ──────────────────────────
    for tool_node in ("sql_node", "rag_node", "web_search_node"):
        graph.add_edge(tool_node, "synthesise_node")

    # ── Linear tail ─────────────────────────────────────────────────
    graph.add_edge("synthesise_node",      "update_history_node")
    graph.add_edge("update_history_node",  END)

    return graph.compile()


def save_graph_visualization(compiled_graph, filepath: str = "agent_graph.png"):
    """
    Save the compiled graph as a Mermaid PNG visualization.

    Args:
        compiled_graph: The compiled StateGraph from build_graph()
        filepath: Path where to save the PNG file (default: agent_graph.png)
    """
    try:
        graph_obj = compiled_graph.get_graph()
        png_data = graph_obj.draw_mermaid_png()
        with open(filepath, "wb") as f:
            f.write(png_data)
        print(f"Graph visualization saved to {filepath}")
    except Exception as e:
        print(f"Error saving graph visualization: {e}")
```



LangGraph Flow for the Usecase

8: Define and Run `setup.py`

The setup script does three things:
1. downloads the Kaggle e-commerce dataset,
2. ingests CSV files into SQLite,
3. builds the FAISS index from PDFs.

Here is the complete `setup.py`

```
#!/usr/bin/env python3
# setup.py
```

```
# ──────────────────────────────────────────────────────────────────────────────
# Run this ONCE before starting the agent.
#
# What it does:
#   1. Downloads the Kaggle dataset (bytadit/ecommerce-order-dataset)
#   2. Loads the train/ CSVs into a local SQLite database
#   3. Builds the FAISS vector index from any PDFs in pdf_docs/
#
# Usage:
#   python setup.py
#   python setup.py --force    # re-ingest even if DB/index already exist
# ──────────────────────────────────────────────────────────────────────────────

import argparse
import os
import sys

# Ensure project root is on the path when running directly
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from data_loader import download_and_ingest, list_tables
from tools.rag_tool import build_index
from config import PDF_DIR, DB_PATH, FAISS_INDEX_DIR


def main(force: bool = False) -> None:
    print("=" * 60)
    print("  E-Commerce Agent — Setup")
    print("=" * 60)

    # ── Step 1: SQLite DB ─────────────────────────────────────────────────────
    print("\n[1/2] Ingesting Kaggle dataset into SQLite …")
    download_and_ingest(force=force)
    print(f"      Tables in DB: {list_tables()}")

    # ── Step 2: FAISS index ───────────────────────────────────────────────────
    print(f"\n[2/2] Building FAISS index from PDFs in '{PDF_DIR}' …")
    os.makedirs(PDF_DIR, exist_ok=True)
    pdf_count = len([f for f in os.listdir(PDF_DIR) if f.lower().endswith(".pdf'
    if pdf_count == 0:
        print(f"      No PDFs found in {PDF_DIR}.")
        print("      Add PDF files there and re-run: python setup.py --force")
        print("      The agent will still work for SQL and web-search routes.")
    build_index(force=force)

    print("\n" + "=" * 60)
    print("  Setup complete!  Run the agent with:  python main.py")
    print("=" * 60 + "\n")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="E-Commerce Agent setup")
    parser.add_argument(
```

```
        "--force", action="store_true",
        help="Re-ingest data and rebuild FAISS index even if they already exist.
    )
    args = parser.parse_args()
    main(force=args.force)
```

In the terminal, please run

```
python setup.py
```

It will create the `data/ecommerce.db` dataset and `data/faiss_index/index.faiss` for vector db in the repo.

9. Finally main.py

The `main.py` file serves as the entry point for the application, providing an interactive CLI and demo mode to interact with the GenAI agent while initializing sessions and validating required configurations.

- Supports both interactive and scripted demo modes to test SQL, RAG, and web search paths
- Handles session management, environment validation, and graph visualization for the LangGraph pipeline

```
#!/usr/bin/env python3
# main.py
# ──────────────────────────────────────────────────────────────────────────────
# Interactive CLI for the E-Commerce Routing Agent.
#
# Prerequisites:
#   1. python setup.py            (download data + build index)
#   2. export OPENAI_API_KEY=...
#   3. export SERPER_API_KEY=...
#
# Usage:
#   python main.py               # interactive mode
#   python main.py --demo        # run a scripted multi-turn demo
# ──────────────────────────────────────────────────────────────────────────────

import argparse
```

```
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from session import EcommerceSession
from config import OPENAI_API_KEY, SERPER_API_KEY
from agent.graph import save_graph_visualization


def _check_env() -> None:
    missing = []
    if not OPENAI_API_KEY:
        missing.append("OPENAI_API_KEY")
    if not SERPER_API_KEY:
        missing.append("SERPER_API_KEY")
    if missing:
        print(f"[ERROR] Missing environment variables: {', '.join(missing)}")
        print("Set them before running:")
        for var in missing:
            print(f"  export {var}=your_key_here")
        sys.exit(1)


def run_demo(session: EcommerceSession) -> None:
    """
    Run a scripted multi-turn demo covering all three routing paths.
    """
    demo_turns = [
        # ── SQL queries ──────────────────────────────────────────────
        "How many orders were delivered successfully?",
        #"What are the top 5 customer states by total number of orders?",
        #"What is the average payment value for credit card transactions?",
        #"Show me the top 5 product categories by total revenue.",
        #"Which payment type is most commonly used?",

        # ── Follow-up (multi-turn context test) ──────────────────────
        #"Now show me those same categories but only for delivered orders.",

        # ── Web search ───────────────────────────────────────────────
        "What are the latest e-commerce trends in Brazil for 2024?",

        # ── RAG (PDF documents) ──────────────────────────────────────
        "What does our return policy say about electronics?",
    ]

    print("\n" + "=" * 60)
    print("  DEMO MODE — Scripted Multi-Turn Conversation")
    print("=" * 60)

    for i, question in enumerate(demo_turns, 1):
        print(f"\n{'─'*60}")
        print(f"[Turn {i}] USER: {question}")
```

```
            print("—" * 60)
            answer = session.ask(question)
            print(f"\nASSISTANT:\n{answer}")

    print("\n" + "=" * 60)
    print("  Demo complete.")
    print("=" * 60)
    session.print_history()


def run_interactive(session: EcommerceSession) -> None:
    """
    REPL-style interactive session.
    """
    print("\n" + "=" * 60)
    print("  E-Commerce Analytics Agent")
    print("  Routes: SQL  |  RAG (PDFs)  |  Web Search")
    print("  Type 'quit' or 'exit' to stop.")
    print("  Type 'reset' to clear conversation history.")
    print("  Type 'history' to print the full conversation.")
    print("=" * 60 + "\n")

    while True:
        try:
            user_input = input("You: ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nGoodbye!")
            break

        if not user_input:
            continue

        if user_input.lower() in ("quit", "exit"):
            print("Goodbye!")
            break

        if user_input.lower() == "reset":
            session.reset()
            continue

        if user_input.lower() == "history":
            session.print_history()
            continue

        print()
        answer = session.ask(user_input)
        print(f"\nAssistant:\n{answer}\n")


def main() -> None:
    _check_env()

    parser = argparse.ArgumentParser(description="E-Commerce Routing Agent")
```

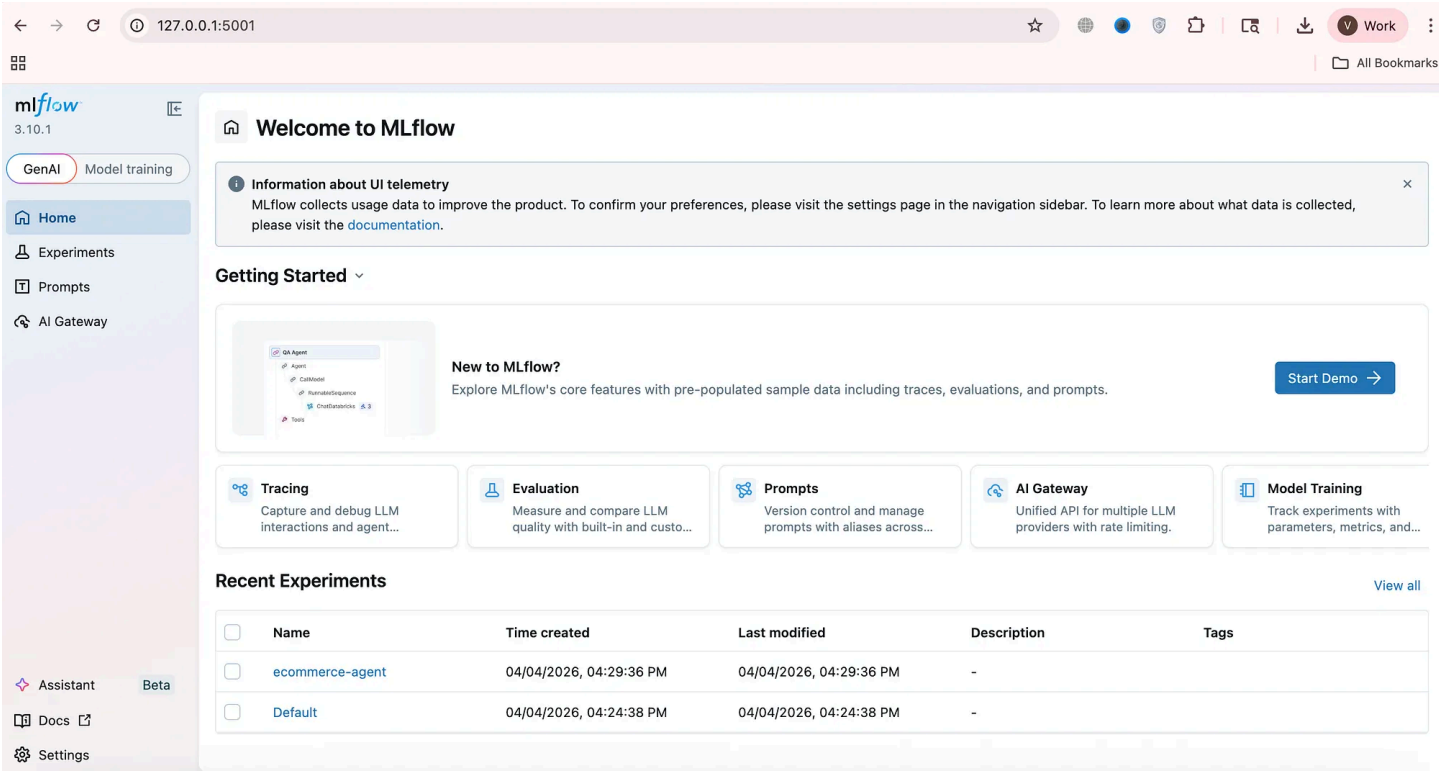
```
    parser.add_argument(
        "--demo", action="store_true",
        help="Run a scripted multi-turn demo instead of interactive mode."
    )
    args = parser.parse_args()

    session = EcommerceSession()
    print(f"[main] Session ID: {session.session_id}")

    # Save graph visualization
    save_graph_visualization(session.graph)

    if args.demo:
        run_demo(session)
    else:
        run_interactive(session)


if __name__ == "__main__":
    main()
```

10. Running the Application

First please run below command in the terminal to activate mlflow

```
mlflow server --host 0.0.0.0 --port 5001
```

This will open a mlflow UI at thisurl: http://127.0.0.1:5001/ :



MLFlow UI Screen (Image by Author)

After that, please run below command in the terminal.

```
python main.py --demo
```

This will run 3 queries automatically and all those will be logged in the MLFLow. As you can see, each request is fully traceable in MLflow UI, exposing every decision, tool call, and LLM interaction.

[Turn 1] USER: How many orders were delivered successfully?

- SQL

[Turn 2] USER: What are the latest e-commerce trends in Brazil for 2024?

- Websearch

[Turn 3] USER: What does our return policy say about electronics?

- RAG

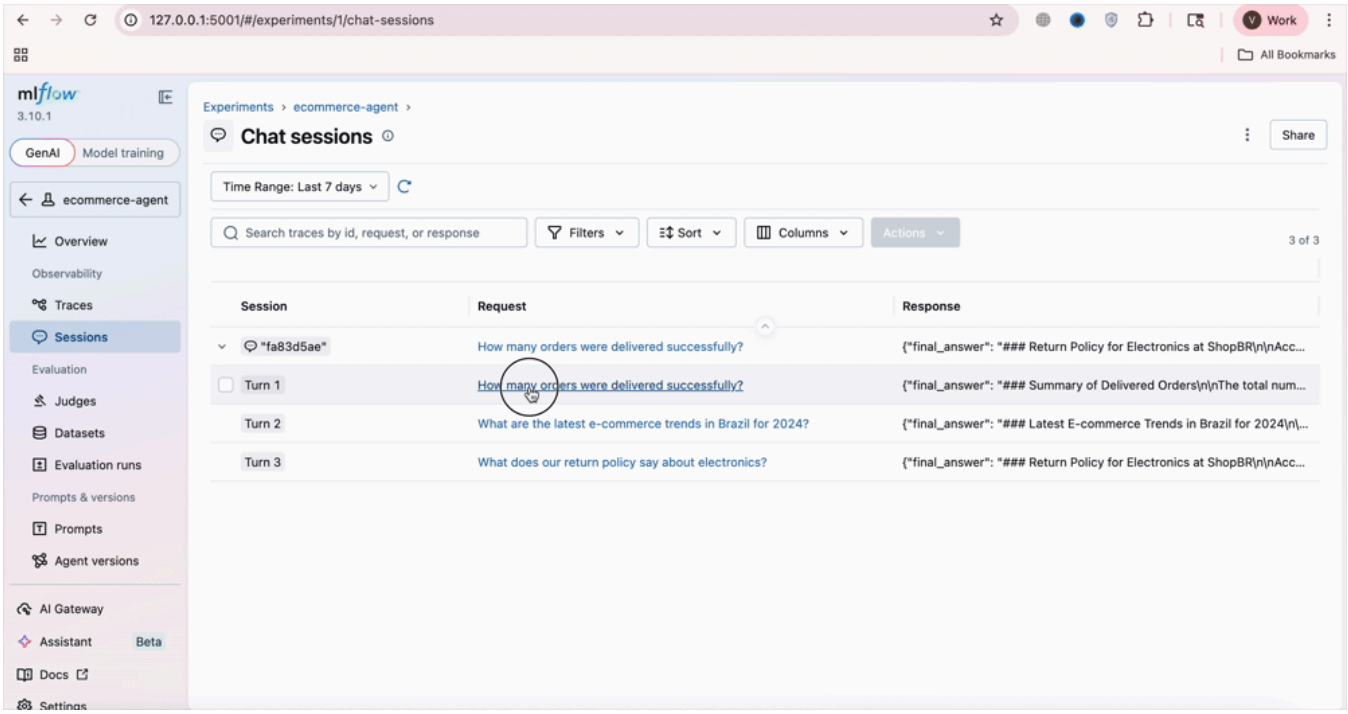
Here is the terminal output at my end:

```
Graph visualization saved to agent_graph.png

============================================================
  DEMO MODE — Scripted Multi-Turn Conversation
============================================================

------------------------------------------------------------
[Turn 1] USER: How many orders were delivered successfully?
------------------------------------------------------------

[router] → SQL  |  The question asks for a specific count of orders, which is do
[sql_node] Generating and executing SQL …
[sql_node] SQL: SELECT COUNT(o.order_id) AS delivered_orders_count
FROM orders AS o
WHERE o.order_status = 'delivered'
LIMIT 50;
[sql_node] 1 rows returned.
[synthesise_node] Answer ready (279 chars).

ASSISTANT:
### Summary of Delivered Orders

The total number of orders that were successfully delivered is **87,428**.

This figure indicates the volume of completed transactions where the order statu

------------------------------------------------------------
[Turn 2] USER: What are the latest e-commerce trends in Brazil for 2024?
------------------------------------------------------------

[router] → WEB_SEARCH  |  The question asks about current events and trends in t
[web_node] Searching the web …
[web_node] 5 results for query: 'What are the latest e-commerce trends in Brazil
[synthesise_node] Answer ready (1621 chars).

ASSISTANT:
### Latest E-commerce Trends in Brazil for 2024

1. **Market Size and Growth**:
   - Brazil's retail e-commerce market is projected to reach **$81.74 billion**
   - The market is expected to grow at a **CAGR of 30%** through 2024, indicatin

2. **Market Share**:
   - E-commerce in Brazil is anticipated to represent around **28.5%** of the to

3. **Future Projections**:
   - Revenue in the Brazilian e-commerce market is expected to reach **$48.26 bi
   - The B2C e-commerce market is forecasted to grow at a **CAGR of 17.7%** from
```

```
These trends highlight a thriving e-commerce landscape in Brazil, characterized

──────────────────────────────────────────────────────────────────────

[Turn 3] USER: What does our return policy say about electronics?
──────────────────────────────────────────────────────────────────────

[router] → RAG  |  The question asks for specific information about the return p
[rag_node] Retrieving document chunks …
[rag_tool] Loading embedding model: sentence-transformers/all-MiniLM-L6-v2 …
Loading weights: 100%|██████████████████████████████| 103/103 [00:00<00:00, 7
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; no
[rag_node] 4 chunks retrieved from: ['shopbr_return_policy.pdf']
[synthesise_node] Answer ready (1158 chars).

ASSISTANT:
### Return Policy for Electronics at ShopBR

According to the ShopBR return policy, the following points are highlighted rega

1. **Quality Standards**:
   - Electronics are subject to heightened quality standards due to their comple

2. **Return Eligibility**:
   - Electronics can be returned if they are defective or damaged upon arrival.
   - Returns for electronics that have been tampered with, misused, or physicall

3. **Replacement and Refund**:
   - If an electronic item is defective or damaged on arrival, customers are el

4. **Authorized Service Centers**:
   - ShopBR partners with authorized service centers to ensure genuine replaceme

5. **Return and Replacement Windows**:
   - Specific return and replacement windows may apply, which are detailed in th

This policy is effective as of **January 1, 2024**, and complies with the Brazil

_Sources: shopbr_return_policy.pdf_

══════════════════════════════════════════════════════════════════════
  Demo complete.
══════════════════════════════════════════════════════════════════════

══════════════════════════════════════════════════════════════════════
  Session fa83d5ae  |  3 turns
══════════════════════════════════════════════════════════════════════
```

```
[User]
How many orders were delivered successfully?

[Assistant]
### Summary of Delivered Orders

The total number of orders that were successfully delivered is **87,428**.

This figure indicates the volume of completed transactions where the order statu

[User]
What are the latest e-commerce trends in Brazil for 2024?

[Assistant]
### Latest E-commerce Trends in Brazil for 2024

1. **Market Size and Growth**:
   - Brazil's retail e-commerce market is projected to reach **$81.74 billion**
   - The market is expected to grow at a **CAGR of 30%** through 2024, indicatir

2. **Market Share**:
   - E-commerce in Brazil is anticipated to represent around **28.5%** of the to

3. **Future Projections**:
   - Revenue in the Brazilian e-commerce market is expected to reach **$48.26 bi
   - The B2C e-commerce market is forecasted to grow at a **CAGR of 17.7%** fron

These trends highlight a thriving e-commerce landscape in Brazil, characterized

[User]
What does our return policy say about electronics?

[Assistant]
### Return Policy for Electronics at ShopBR

According to the ShopBR return policy, the following points are highlighted rega

1. **Quality Standards**:
   - Electronics are subject to heightened quality standards due to their comple

2. **Return Eligibility**:
   - Electronics can be returned if they are defective or damaged upon arrival.
   - Returns for electronics that have been tampered with, misused, or physicall

3. **Replacement and Refund**:
   - If an electronic item is defective or damaged on arrival, customers are el

4. **Authorized Service Centers**:
   - ShopBR partners with authorized service centers to ensure genuine replaceme

5. **Return and Replacement Windows**:
   - Specific return and replacement windows may apply, which are detailed in th
```

```
This policy is effective as of **January 1, 2024**, and complies with the Brazil

_Sources: shopbr_return_policy.pdf_
```



ML Flow Screen Recording (Video by Author)

This completes our case study on building a fully observable GenAI pipeline demonstrating how MLflow transforms complex, multi-step systems into transparent, debuggable, and production-ready applications.

Conclusion

Building GenAI systems is no longer just about getting the right answer. It's about **understanding how that answer was produced.** As pipelines evolve into multi-step, agentic workflows involving retrieval, reasoning, and external tools, traditional logging falls short. You need visibility not just into *what* happened, but *why* it happened.

This is where MLflow's observability truly shines. By combining **spans, traces, and evaluations**, you gain a complete, structured view of your system from individual LLM calls to full request lifecycles and quality metrics. In our Text2SQL + RAG + Web

Search pipeline, this transforms debugging from guesswork into a precise, data-driven process.

Ultimately, observability is what turns a GenAI system from a fragile prototype into a **production-ready, trustworthy application**. Because in the real world, it's not enough for your pipeline to work — you need to **see how it works, measure how well it performs, and fix it when it doesn't.**

References:

1. Kaggle: Ecommerce Order & Supply Chain Dataset
2. Alpha Iterations: GenAI Observability
3. MLFlow GenAI Tutorials
4. MLFlow Github

Thank you for reading the article.

GenAI is complex and chaotic but getting started doesn't have to be. I focus on making that first step simpler for you. Follow along for regular updates and more such articles.

Feel free to connect on Linkedin if you're on a similar path.

And if you're still curious, there's more to explore.

1. Build Agentic RAG using LangGraph
2. Practical Guide to Using ChromaDB for RAG and Semantic Search
3. Reading Images with GPT-4o: The Future of Visual Understanding with AI
4. Agentic AI Project: Build Mini Perplexity AI Chatbot : Step by Step Guide [Code Included]
5. Agentic AI: Build ReAct Agent using LangGraph
6. Agentic AI Project: Build a multi-agent system with LangGraph and OpenAI API

7. Building an AI Agent with Model Context Protocol (MCP): A Complete Guide
8. TOON vs JSON: A Comprehensive Performance Comparison
9. Building an Intelligent Resume Transformation Agent Powered by LangGraph and gpt-4o-mini
10. Agentic AI Project: Build a Customer Service Chatbot for a Clinic
11. Vectorless RAG: How I Built a RAG System Without Embeddings, Databases, or Vector Similarity
12. Agentic AI Project: Build AI Agents to chat with YouTube Videos

Agentic Ai | Mlflow | Langgraph | Llm Observability | Retrieval Augmented Gen

Following

Published in Towards AI

139K followers · Last published 1 day ago

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: https://email-course.towardsai.net/

Follow

Written by Alpha Iterations

1.2K followers · 29 following

Generative AI | Agentic AI | AI Applications